Tech Stack, Build Process & Deployment Guide

Clothing Mill Production & Wage Management System

This picks up where the feature plan left off. Since this is a 2-user internal tool (Admin + Manager) with no public traffic, the guiding principle throughout is: **pick the smallest, free/cheapest option that does the job properly** — no paid APIs, no enterprise infrastructure, nothing that needs ongoing subscription cost unless it's genuinely necessary.

---

1. The Tech Stack (and why)

Layer	Choice	Why this, not something fancier
Frontend + Backend	**Next.js** (React framework)	Handles both the UI and the server-side logic (API routes) in one project. You don't need a separate backend server — this alone removes a whole category of complexity.
Styling	**Tailwind CSS**	Fast to write, no separate CSS files to manage, easy to make print-friendly invoice layouts.
Database	**PostgreSQL**	Reliable, handles concurrent writes properly (important since two roles may be entering/viewing data at once), and every serious hosting option supports it for free at small scale.
Database Access	**Prisma** (ORM)	Lets you define your tables (Machine, Worker, ProductionEntry, etc.) as code, auto-generates safe queries, and handles migrations cleanly — much less error-prone than writing raw SQL by hand for a solo/small-team project.
Authentication	**Auth.js (NextAuth) with Credentials provider**	Free, self-hosted, no third-party account needed. Since there are only ever two accounts (Admin, Manager), you don't need Google/Facebook login or a paid auth service — just username + password, hashed with bcrypt.

Printing/Invoices	Plain CSS `@media print` stylesheet	No library or API needed. The browser's own "Print" / "Print to PDF" covers both the print and download requirement.
Charts (optional, Phase 4)	**Recharts**	Free, lightweight, works natively with React. Only needed if you build the optional monthly trend chart.
Hosting (app)	**Vercel** (free tier)	Built by the same team as Next.js, deployment is a `git push` away, free tier is genuinely sufficient for 2 internal users.
Hosting (database)	**Neon** (free tier Postgres)	Free tier covers far more storage/rows than a single mill's monthly production data will ever produce. Works seamlessly with Vercel.

APIs you do *not* need to pay for or integrate: payment gateways, SMS/email services, maps, third-party auth, PDF-generation APIs, analytics services. None of the confirmed requirements call for these. If a need comes up later (e.g., the client wants an SMS alert when an entry is missed for the day), that can be added then — no reason to build it now.

Total recurring cost at this scale: 0 BDT, unless you choose to buy a custom domain (covered in Section 5) — everything else fits comfortably in free tiers for two users and modest data volume.

---

2. Local Development First — Yes, Build Locally Before Deploying

Always build and test locally first, then deploy. This isn't optional for a project like this — it's the only sane way to work. The flow is:

1. **Build and test on your own machine**, using a free *development* database (a separate Neon project/branch, not the same one the client will eventually use).
2. **Commit your code to Git** and push to **GitHub** (free, private repo is fine) as you go — this is your version history and also what Vercel will deploy from.
3. **Deploy to Vercel**, connected directly to your GitHub repo, pointing to a separate *production* database.
4. Keep local (dev) and live (production) databases **completely separate** so your testing/experimenting never touches real production data once the client starts using it.

This local → GitHub → Vercel pipeline means every time you push a change, it auto-deploys — no manual upload steps once it's set up.

---

3. Step-by-Step Build Process

Step 1 — Environment Setup

- Install **Node.js** (LTS version) on your machine.
- Install **Git**, create a GitHub account if you don't have one, and create a new private repository for this project.
- Create a free account on **Neon** (database) and **Vercel** (hosting) — both just need an email/GitHub login, no payment info required for free tier.

Step 2 — Scaffold the Project

- Create the Next.js project (with Tailwind and TypeScript enabled — TypeScript is worth the small learning curve here since it catches data-model mistakes, like passing a string where a yard total should be a number, before they cause a wrong wage calculation).
- Install Prisma, Auth.js, and bcrypt.

Step 3 — Define the Database Schema

Translate the entities from the feature plan directly into a Prisma schema: `Machine`, `Worker`, `WageRate`, `ProductionEntry` (with its segment list and computed total), and `User` (for the two accounts, with a `role` field of `ADMIN` or `MANAGER`). Run a migration to create these tables in your dev database.

Step 4 — Authentication

Build a simple login page. Store only **hashed** passwords (bcrypt) — never plain text, even for a 2-user system. Use Auth.js sessions so the rest of the app knows who's logged in and what role they have, and protect Admin-only pages (wage rate settings, user management) with a role check.

Step 5 — Build Features in the Phase Order from the Feature Plan

1. Machines & Workers management (basic CRUD pages)
2. Daily Production Entry screen
3. Wage rate settings page
4. Calculation logic (as a shared function, not duplicated across pages, so the math lives in exactly one place)
5. Daily & Monthly reports
6. Worker payslip / mill summary invoice, with print stylesheet
7. Dashboard

8. Polish pass (loading states, error messages, mobile responsiveness for the entry screen)

Step 6 — Test Thoroughly Before Going Live

Specifically test: a worker logged on two machines in one day (totals should combine correctly), a month boundary (30 vs. 31 day months, February), a wage rate change mid-month (old entries should still use the old rate), and the print layout on actual paper (not just on-screen) to check it doesn't cut off oddly.

Step 7 — Deploy

Push the finished code to GitHub, connect the repo to Vercel, set the production environment variables (database connection string, auth secret — see Section 6), and deploy. Run the Prisma migration against the production database once, to create the real tables.

Step 8 — Hand Off

Create the actual Admin and Manager accounts for the client (don't ship with a default/test password), do a walkthrough with the client on the live site, and document the login URL and credentials somewhere safe for them.

---

4. Project Structure (so it stays maintainable)

```
mill-system/
├── prisma/
│   └── schema.prisma       # Database schema (Machine, Worker, ProductionEntry, etc.)
├── src/
│   ├── app/                # Pages (Next.js App Router): login, dashboard, entry, reports, etc.
│   ├── components/         # Reusable UI pieces (forms, tables, print layout)
│   ├── lib/
│   │   ├── auth.ts         # Auth.js config
│   │   ├── db.ts           # Prisma client setup
│   │   └── wage-calc.ts    # The one shared place wage/yard math lives
│   └── middleware.ts       # Route protection (Admin vs Manager access)
├── .env.local              # Local secrets (never committed to Git)
└── package.json
```

Keeping the wage/yard calculation logic in one shared file (`wage-calc.ts`) rather than copy-pasted into each report/invoice page is the single most important structural decision — it guarantees the daily report, monthly report, and payslip can never disagree with each other.

---

5. Domain — Optional, Cheap if You Want One

You don't strictly need a custom domain. Vercel gives every project a free `yourproject.vercel.app` URL, which is enough for a 2-user internal tool — the client just bookmarks it.

If the client wants something like `millsystem.com` for a more professional feel:

- Buy it through a registrar like **Namecheap** or **Google Domains' successor (Squarespace Domains)** — typically **$10–15/year** for a `.com`.
- A `.com.bd` domain (Bangladesh-specific) is also an option, registered through BTCL-accredited local registrars, usually similarly priced or slightly cheaper.
- Either way, point the domain at Vercel (a quick DNS settings change) — Vercel's free tier supports custom domains.

This is the **only cost** in the entire stack, and it's optional.

---

6. Environment Variables & Secrets (must-do, not optional)

Set these in Vercel's project settings (not committed to GitHub):

- `DATABASE_URL` — the production Neon connection string
- `AUTH_SECRET` — a random secret string used by Auth.js to sign sessions (generate this once, keep it private)

Locally, the same variables go in a `.env.local` file that's excluded from Git via `.gitignore` (Next.js sets this up by default). This is the standard way secrets are handled — they should never appear in your GitHub repo's code.

---

7. Security Checklist (small system, but still worth doing right)

- Passwords hashed with bcrypt, never stored in plain text.
- HTTPS is automatic on Vercel — nothing to configure.

- Admin-only routes (wage settings, user management) checked server-side, not just hidden in the UI — a Manager shouldn't be able to reach those pages by typing the URL directly.
- Input validation on the entry form (e.g., a library like Zod) to reject obviously invalid data (negative yards, impossible dates) before it's saved.
- Regular database backups — Neon's free tier includes automatic point-in-time recovery for a short window; for extra safety, an occasional manual export (`pg_dump`) kept somewhere safe costs nothing and takes a minute.

---

8. Cost Summary

Item	Cost
Next.js, Tailwind, Prisma, Auth.js, bcrypt, Recharts	Free (open source)
Vercel hosting (free tier)	0 BDT/month
Neon database (free tier)	0 BDT/month
GitHub (private repo)	Free
Custom domain (optional)	~$10–15/year only if the client wants one

Bottom line: this can be built and run at essentially zero ongoing cost, with the only possible expense being an optional domain name.

---

9. Suggested Order to Tackle This Document

Read Sections 1–2 to understand the stack and workflow, then follow Section 3 top to bottom as your literal build checklist. Sections 4–8 are reference material to come back to as you hit each relevant step (project structure when you scaffold, domain when you're ready to go live, security checklist right before handoff).